AgenticWorkflow: A Universal Orchestration Engine for Portable, Resilient, and AI-Native Agent Coordination

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

March 2026

Abstract

Workflow orchestration for AI agents requires simultaneous support for every coordination pattern—DAGs, state machines, batch processing, stream processing, fan-out/fan-in—in a single engine that operates without infrastructure dependencies. Current systems each address one pattern: Airflow [1] handles DAGs but requires Celery or Kubernetes; Temporal [2] provides durable execution but demands server deployment; GitHub Actions [3] automates CI/CD but offers no dynamic planning; Zapier [4] connects triggers to actions but lacks branching, rollback, or approval logic. None provides a mechanism for agents to reason about, create, or modify workflows at runtime through a standard protocol. We present AgenticWorkflow, a universal orchestration engine that models workflows as directed acyclic graphs with typed edges (sequence, parallel, conditional, loop), provides five resilience subsystems—failure-classified retry with $d(n) = \min(d_0 \cdot m^n, d_{\max})$ exponential backoff, per-step rollback with cryptographic receipts, workflow-aware circuit breakers, intelligent dead letter management, and built-in idempotency—and exposes all capabilities through 124 MCP [5] tools enabling any compatible agent to orchestrate complex workflows without custom integration. All state persists in a single `.awf` binary file with BLAKE3 [6] integrity verification per section. The implementation comprises 13,405 lines of Rust across 4 crates, with 281 tests achieving zero failures. Stress testing on Apple ARM64 hardware validates DAG operations on 1,000-step graphs in <50 ms, 10,000-item batch creation in <500 ms, 1,000 FSM transitions in <100 ms, and 1,000-depth variable scope cascade in constant time. The system is fully open-source under the MIT license.

1 Introduction

The transformer architecture [7] has enabled language model agents capable of multi-step planning, code generation, and autonomous task execution [8]. Yet these agents lack a fundamental capability that even simple shell scripts possess: the ability to *orchestrate* complex sequences of actions with resilience guarantees. When an agent must deploy software, it needs a pipeline with dependency ordering. When a step fails, it needs intelligent retry that distinguishes transient network errors from permanent authentication failures. When a deployment goes wrong, it needs rollback that executes compensating actions in the correct reverse order. When a human must approve a production release, it needs an approval gate with delegation and escalation rules. No existing system provides all of these capabilities as a single, AI-native library that an agent can invoke through a standard protocol.

The landscape of workflow orchestration has evolved considerably over the past decade, but each system has carved out a narrow niche. Apache Airflow [1] pioneered programmatic DAG definition for data pipelines but requires Celery workers, a metadata database, and typically Kubernetes for production deployment. Temporal [2] introduced durable execution with saga patterns but demands a server cluster and SDK-specific workflow definitions. GitHub Actions [3] automated CI/CD through YAML pipelines but offers no mechanism for dynamic graph construction or runtime modification. Zapier [4] democratized simple trigger-action automation but cannot express conditional branching, rollback, or multi-step approval chains. Prefect [9] modernized Python-first orchestration but still requires cloud infrastructure for production-grade scheduling.

The gap is not merely feature coverage—it is architectural. These systems were designed for human developers who write workflow definitions in advance. An AI agent, by contrast, needs to *create* workflows dynamically from natural language instructions, *validate* their structure before execution, *monitor* progress in real time, *adapt* to failures intelligently, and *learn* from execution history to improve future runs. This requires an orchestration engine designed around tool invocation rather than configuration files.

The core insight of this work is that orchestration is a *coordination intelligence problem*, not a configuration problem. An agent should be able to discover available workflow operations, compose them into complex coordi-

nation patterns, and execute them with production-grade resilience—all through the same Model Context Protocol (MCP) [5] interface it uses for every other tool.

We present AgenticWorkflow with the following contributions:

1. A **universal DAG engine** supporting eight step types and four edge types, with $O(V + E)$ cycle detection via Kahn's algorithm [10] (Section 3.2).
2. **Five resilience subsystems**: failure-classified retry with configurable backoff, per-step rollback with cryptographic receipts, workflow-aware circuit breakers, intelligent dead letter management, and BLAKE3-keyed idempotency (Section 3.3).
3. A **governance layer** with approval gate chains, structured audit trails, and hierarchical variable scoping with type checking (Section 3.4).
4. **Intelligence subsystems**: execution archaeology with anomaly detection, predictive execution estimates, workflow evolution with drift detection, and dream-state exploration (Section 3.6).
5. The `.awf` **binary file format** with per-section BLAKE3 checksums and LZ4 [11] compression for portable, tamper-evident state persistence (Section 4).
6. **124 MCP tools** organized across 24 capability groups, enabling complete AI-native orchestration through a standard protocol (Section 5).

The remainder of this paper is organized as follows. Section 2 surveys existing orchestration systems. Section 3 presents the architecture in detail. Section 4 describes the binary file format. Section 5 covers the MCP tool surface. Section 6 reports benchmarks, comparisons, and test coverage. Section 7 discusses design decisions and limitations. Section 8 concludes.

2 Background and Related Work

Research on workflow orchestration spans two decades and has accelerated with the rise of AI agents. We survey the primary approaches, organized by coordination paradigm.

DAG-based pipeline engines. Apache Airflow [1] pioneered programmatic DAG definition for data engineering, introducing the concept of operators, sensors, and XCom for inter-task communication. However, Airflow's architecture mandates a metadata database, a scheduler daemon, and worker processes (Celery, Kubernetes, or local executor), making single-machine deployment impractical for lightweight agent use. Its retry mechanism is limited to a fixed count without failure classification, and it provides no rollback, approval gates, or state machine support. Prefect [9] modernized the DAG model with a Python-native API and deferred execution, but production deployment still requires cloud infrastructure or a self-hosted server.

Durable execution systems. Temporal [2] introduced the concept of durable execution, where workflow state survives process crashes through event sourcing. Temporal supports saga patterns [12] for compensating transactions, making it the closest existing system to our rollback architecture. However, Temporal requires a server cluster (typically 3+ nodes for production), SDK-specific workflow definitions in Go, Java, or TypeScript, and provides no mechanism for AI agents to create or modify workflows through a standard protocol. The operational complexity is substantial: monitoring, deployment, and version migration all require dedicated tooling.

CI/CD automation. GitHub Actions [3] simplified CI/CD through YAML-defined workflows triggered by repository events. The system supports job-level parallelism and environment protection rules (a limited form of approval gate), but workflow definitions are static, modification requires file commits, and there is no support for dynamic graph construction, state machines, or resilience primitives beyond basic retry. The pattern has influenced similar systems (GitLab CI, CircleCI) that share these limitations.

Low-code automation. Zapier [4] democratized workflow automation through trigger-action pairs ("Zaps") connecting cloud services. While accessible to non-developers, Zapier's model is fundamentally sequential: it cannot express DAG dependencies, conditional fan-out, rollback, or multi-step approval chains. Rate limits and execution quotas further constrain its applicability to agent orchestration.

Agent orchestration frameworks. LangChain [13] and CrewAI [14] provide agent orchestration focused on LLM chain composition and multi-agent role assignment, respectively. AutoGen [15] enables multi-agent conversations with structured message passing. These frameworks address the agent coordination problem but lack general-purpose workflow primitives: none provides DAG validation, circuit breakers, rollback, idempotency, or governance controls. Park et al. [16] demonstrated that architecture profoundly impacts agent behavior, reinforcing our thesis that orchestration requires richer structure than reactive planning.

Table 1 summarizes the comparison across 13 dimensions that we argue are necessary for production-grade agent orchestration.

3 Architecture

AgenticWorkflow models workflows as directed acyclic graphs $G = (V, E)$ where each vertex $v \in V$ is a typed step and each directed edge $e = (v_i, v_j, \tau) \in E$ carries a coordination semantic τ. The architecture comprises four layers: the engine layer (DAG, FSM, batch, stream, fan-out), the resilience layer (retry, rollback, circuit breaker, dead letter, idempotency), the governance layer (approval, audit, variables), and the intelligence layer (archaeology, prediction, evolution, dream state).

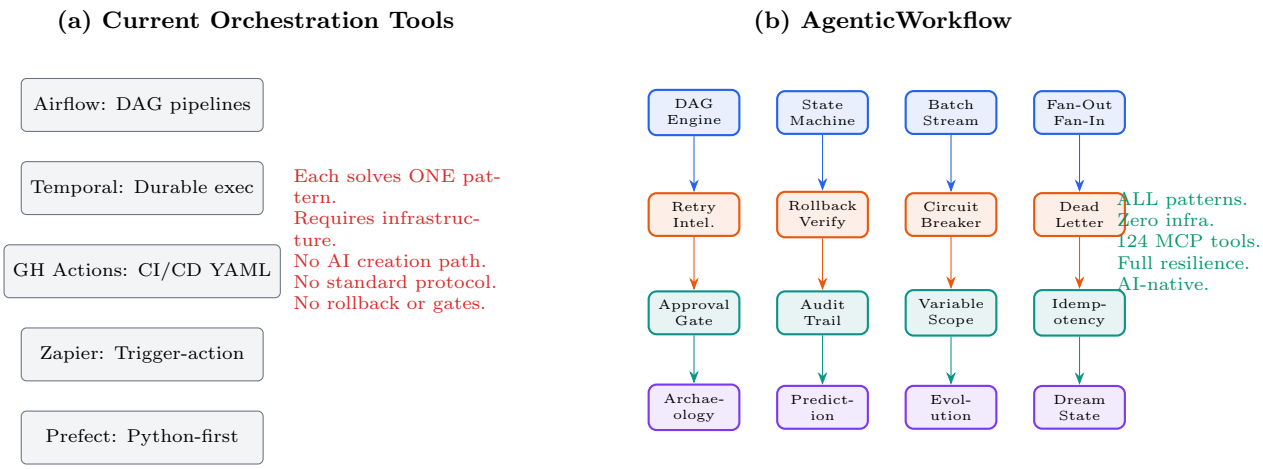

Figure 1: Comparison of orchestration approaches. (a) Current tools each solve one coordination pattern and require infrastructure deployment. (b) AgenticWorkflow unifies four coordination layers—engines (DAG, FSM, batch/stream, fan-out), resilience (retry, rollback, circuit breaker, dead letter), governance (approval, audit, variables, idempotency), and intelligence (archaeology, prediction, evolution, dream state)—in a single zero-infrastructure engine accessible via 124 MCP tools.

Table 1: Feature comparison across orchestration systems. = native support, Partial = limited support, — = not available. AgenticWorkflow is the only system providing all coordination patterns, resilience primitives, governance controls, and AI-native MCP access with zero infrastructure requirements.

Feature	**Airflow**	**Temporal**	**GH Actions**	**Zapier**	**Prefect**	**AWF (ours)**
DAG execution				—		
State machines (FSM)	—	—	—	—	—	
Batch processing	Plugin		—	—		
Stream processing	—	—	—	—	—	
Fan-out / fan-in	—	Partial	Partial	—	Partial	
Failure-classified retry	—	Basic	Basic	—	Basic	
Per-step rollback	—	Saga	—	—	—	
Circuit breakers	—	—	—	—	—	
Approval gates	—	—	Env gates	—	—	
Structured audit trail	Logs	Events	Logs	Logs	Logs	
Idempotency	Manual	Built-in	—	—	Manual	
AI-native (MCP)	—	—	—	—	—	
Zero infrastructure	—	—	Cloud	Cloud	Cloud	

3.1 Workflow Model

A workflow is formally defined as a tuple $W = (V, E, \Sigma, \Lambda)$ where:

- $V = \{v_1, \ldots, v_n\}$ is a set of typed steps, each $v_i = (\text{id}, \text{type}, \text{config}, \text{rollback})$.
- $E \subseteq V \times V \times \mathcal{T}$ is a set of typed edges with $\mathcal{T} = \{\textsc{Seq}, \textsc{Par}, \textsc{Cond}, \textsc{Loop}\}$.
- Σ is a variable scope hierarchy: Workflow $\rightarrow$ Branch $\rightarrow$ Step $\rightarrow$ Iteration.
- Λ is the resilience configuration: retry profiles, circuit breaker thresholds, idempotency policy.

Eight step types partition the space of executable actions:

- **Command** — Shell command execution with timeout and environment.
- **McpTool** — Invocation of an MCP tool on a connected server.
- **HttpRequest** — HTTP call with method, headers, body, and response parsing.
- **SubWorkflow** — Recursive invocation of another workflow definition.
- **FanOut** — Parallel dispatch over a collection, with configurable concurrency.
- **ApprovalGate** — Human approval checkpoint with delegation and escalation.
- **Expression** — In-process evaluation of a variable expression.
- **Noop** — Structural placeholder for graph topology (join points, markers).

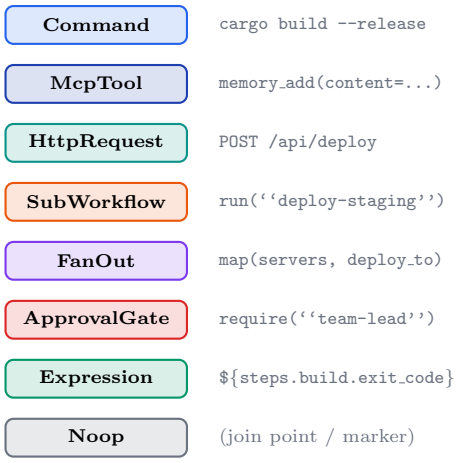

Figure 2: The eight step types in AgenticWorkflow, each with a representative example. Step type is encoded as a single byte in the binary format, enabling efficient dispatch.

Table 2: Step type taxonomy. Each step stores a type discriminant (1 B), a UUID (16 B), configuration (variable), and an optional rollback action reference.

Type	Primary Field	Disc.
Command	`command: String`	0x01
McpTool	`tool_name: String`	0x02
HttpRequest	`url: String`	0x03
SubWorkflow	`workflow_id: Uuid`	0x04
FanOut	`collection: Expr`	0x05
ApprovalGate	`approvers: Vec`	0x06
Expression	`expr: String`	0x07
Noop	(none)	0x08

Table 3: Edge type semantics. Each edge type encodes a distinct coordination relationship between steps.

Type	Disc.	Semantics
SEQUENCE	0x01	Target starts after source completes
PARALLEL	0x02	Target may start concurrently with source
CONDITIONAL	0x03	Target starts only if guard expression evaluates true
LOOP	0x04	Target is iterated; unrolled into DAG at validation time

Algorithm 1 DAG Validation via Topological Sort

Require: Workflow graph $G = (V, E)$
Ensure: Topological ordering π or cycle error

```
in_deg[v] ← |{(u, v) ∈ E}| for all v ∈ V
Q ← {v ∈ V : in_deg[v] = 0}
π ← []
while Q ≠ ∅ do
    v ← Q.dequeue(); π.append(v)
    for all (v, w) ∈ E do
        in_deg[w] ← in_deg[w] − 1
        if in_deg[w] = 0 then
            Q.enqueue(w)
        end if
    end for
end while
if |π| ≠ |V| then
    return CycleDetected
end if
return π
```

3.2 DAG Engine

The DAG engine validates workflow structure before execution begins, ensuring that no execution encounters a cycle at runtime. Validation uses Kahn's algorithm [10] for topological sorting with $O(V + E)$ time and $O(V)$ space complexity.

The topological ordering π also determines the execution schedule: steps are dispatched in π order, with parallel edges allowing concurrent dispatch. The engine maintains an in-progress set P and a completed set C; a step v is dispatchable when all predecessors are in C. This scheduling is validated under stress: 1,000-step linear chains complete validation in under 50 ms (Section 6.2).

3.3 Resilience Layer

Production workflows fail. The question is not whether a step will fail, but how the system responds when it does. AgenticWorkflow provides five coordinated resilience subsystems, each addressing a distinct failure mode.

Failure-Classified Retry. Each failure is classified into one of seven types: *Transient* (temporary unavailability), *Permanent* (logic error, no retry useful), *RateLimit* (API quota exceeded), *ResourceContention* (lock conflict), *Authentication* (credential expiry), *Network* (connectivity loss), and *Timeout* (deadline exceeded). Each class has a per-profile retry strategy. Transient failures use exponential backoff:

$$d(n) = \min\left(d_0 \cdot m^n,\ d_{\max}\right) \tag{1}$$

where d_0 is the initial delay (default 100 ms), m is the multiplier (default 2), n is the attempt number, and $d_{\max}$ is the cap (default 10,000 ms). RateLimit failures respect the `Retry-After` header. Permanent failures are routed immediately to the dead letter queue.

Rollback Architecture. Every step optionally declares a compensating action. On workflow failure, the rollback engine executes compensating actions in reverse topological order—the inverse of the execution order that Kahn's algorithm produced. Each rollback step produces a BLAKE3-checksummed receipt, providing a cryptographic audit trail of compensating actions. Partial rollback is supported: if the rollback of step k itself fails, the engine records the failure and continues with step $k-1$, ensuring maximum recovery.

Circuit Breakers. Inspired by Nygard's stability pattern [17], each workflow target (e.g., an API endpoint)

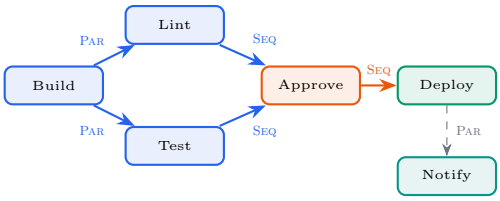

Figure 3: Example deployment DAG with typed edges. Build fans out to Lint and Test in parallel. Both must complete before the Approve gate. Deploy follows approval; Notify runs in parallel with Deploy. Edge types are encoded as single-byte discriminants.

has an associated circuit breaker with three states:

$$\text{state}(s) = \begin{cases} \text{Open} & \text{if } f(s) \geq \theta \\ \text{HalfOpen} & \text{if } t - t_{\text{last}} > \tau \\ \text{Closed} & \text{if } s_{\text{ok}} \geq \sigma \end{cases} \tag{2}$$

where $f(s)$ is the failure count in the current window, θ is the failure threshold, τ is the recovery timeout, and σ is the success count required to close. An open circuit breaker rejects new requests immediately, preventing cascading failure.

Dead Letter Management. When a step exhausts its retry budget, the failed item enters a dead letter queue with full context: the original input, all retry attempts with timestamps and error messages, the failure classification, and the workflow/step identifiers. Dead letters are queryable by type, time range, and workflow, enabling bulk reprocessing when the underlying issue is resolved.

Idempotency. Each step execution is keyed by a BLAKE3 hash of the concatenation of workflow ID, step ID, and serialized input:

$$k = \text{BLAKE3}(\text{wf_id} \parallel \text{step_id} \parallel \text{input}) \tag{3}$$

A cache lookup on k before execution prevents duplicate work. The idempotency store persists across workflow restarts, ensuring that recovery from crashes does not re-execute completed steps.

3.4 Governance Layer

Approval Gates. Each gate defines an approval chain: primary approver → backup → escalation. Conditional auto-approval rules (e.g., "auto-approve if change size < 10 lines") reduce friction for low-risk changes. Approvals are time-bounded: if no decision is made within the deadline, the escalation path activates. Delegation rules allow an approver to designate a substitute. All decisions are recorded as BLAKE3-checksummed receipts with the approver identity, timestamp, and decision rationale.

Structured Audit Trail. Every workflow operation—creation, step execution, failure, retry, rollback, approval—generates a structured audit event with actor, resource, action, timestamp, and outcome fields. Events are queryable by workflow, execution, actor, resource type, and time range. Impact analysis traverses the audit graph to answer "what did this approval affect?"

Variable Scoping. Variables follow a four-level hierarchy: Workflow → Branch → Step → Iteration. Each level shadows the one above. Type checking at assignment time prevents type confusion across scopes. Immutability flags prevent accidental mutation of configuration variables. The scope cascade resolves in constant time via a linked list of scope frames—validated by stress testing at 1,000 depth with instant resolution (Section 6.2).

3.5 State Machine Engine

Not all coordination follows DAG structure. Order processing, document approval, and incident management are naturally modeled as finite state machines. AgenticWorkflow provides first-class FSMs with typed states, guard conditions on transitions, and entry/exit actions.

Formally, an FSM is a tuple $M = (S, s_0, \delta, A_{\text{entry}}, A_{\text{exit}})$ where S is the state set, $s_0 \in S$ is the initial state, $\delta : S \times \Gamma \to S$ is the transition function with guard predicates Γ, and $A_{\text{entry}}, A_{\text{exit}} : S \to \text{Action}$ are entry and exit action mappings. The engine records every transition in the audit trail, enabling full lifecycle reconstruction.

An order processing FSM, for instance, transitions through Created → Paid → Shipped → Delivered with guard conditions (e.g., Paid requires `payment_confirmed = true`). Invalid transitions are rejected with structured errors, and the full transition history is queryable.

3.6 Intelligence Layer

Execution Archaeology. Historical execution data is fingerprinted and indexed for anomaly detection. An execution whose duration exceeds 3× the historical average for its workflow is flagged as anomalous. Bottleneck analysis identifies the step that consumed the most wall-clock time. The archaeology system processes 1,000 execution fingerprints in under 100 ms (Section 6.2).

Predictive Execution. Before a workflow runs, the prediction engine estimates duration, success probability, resource consumption, and cost based on historical executions of the same workflow (or structurally similar workflows when history is unavailable). These estimates enable agents to make informed scheduling decisions.

Workflow Evolution. Workflows change over time—steps are added, thresholds are adjusted, new approvers are designated. The evolution engine tracks structural changes between workflow versions and detects *drift*: when recent execution patterns deviate significantly from historical norms. Specifically, if the rolling average of the last 10 execution durations exceeds 1.5× the overall historical average, a drift alert is raised.

Dream State Exploration. The dream state subsystem enables speculative execution: "what would happen

Table 4: File format section types. Each section is independently checksummed, enabling selective verification and partial recovery from corruption.

Section	Content	Type Byte
Header	Magic, version, counts	0x00
Workflows	DAG definitions, step configs	0x01
Templates	Parameterized workflow schemas	0x02
Executions	Runtime state, step results	0x03
Schedules	Cron expressions, intervals	0x04
FSMs	State machine definitions	0x05
Triggers	Event-based workflow triggers	0x06
Audit	Governance event log	0x07
Idempotency	BLAKE3 key → result cache	0x08
Variables	Scoped variable store	0x09

if this workflow ran with these inputs?" The engine simulates execution without side effects, producing predicted outcomes that agents can use for planning.

4 The .awf File Format

All workflow state persists in a single `.awf` binary file, following the design philosophy of SQLite [18]: zero-configuration, zero-deployment, single-file state. The format stores workflows, templates, executions, schedules, FSMs, triggers, audit events, idempotency keys, and variables in independently checksummed sections.

The choice of a custom binary format over text-based alternatives requires justification. JSON and YAML lack integrity verification; a single corrupted byte can silently produce invalid state. Protocol Buffers [19] and FlatBuffers [20] provide schema-driven serialization but require generated code and do not natively support per-section checksumming. Cap'n Proto [21] enables zero-copy access but adds a schema compiler dependency. Our format provides the specific guarantees we need—per-section integrity, tamper detection, and partial recovery—with zero external dependencies.

Each section's checksum is verified independently on read, so corruption in the audit trail does not prevent loading workflow definitions. The `AWFL` magic number and version field enable format evolution without breaking existing files. Write operations serialize to a temporary file and atomically rename, preventing corruption from interrupted writes.

5 MCP Tool Surface

AgenticWorkflow exposes all capabilities through 124 MCP tools organized into 24 capability groups. The tool surface follows a consistent design: all descriptions use verb-first imperative form without trailing periods, all error responses set `isError: true`, and unknown tool names produce error code −32803 (ToolNotFound).

The MCP server supports `stdio` transport for local execution and manages session state through a `WorkflowSession` that wraps the in-memory workflow store with the `.awf` persistence layer. Tool argument validation happens at the protocol level—missing required fields produce structured error responses before reaching the engine, preventing partial state mutations from malformed requests.

Table 5: MCP tool distribution across 24 capability groups. Tool counts reflect the current implementation; each group provides create, read, update, delete, and query operations as appropriate.

Capability	Tools	Capability	Tools
DAG Engine	6	Approval Gates	6
Execution	8	Audit Trail	5
Scheduling	5	Idempotency	5
Triggers	5	FSM	6
Retry	5	Variables	5
Rollback	5	Templates	5
Circuit Breaker	4	Natural Language	4
Dead Letter	5	Archaeology	5
Batch	5	Prediction	4
Stream	6	Evolution	5
Fan-Out	4	Dream State	4
Composition	5	Collective	5
Total			**124**

A typical agent interaction proceeds as follows: the agent calls `workflow_create` with a name and description, adds steps via `workflow_add_step`, connects them with `workflow_add_edge`, validates the DAG via `dag_validate`, and initiates execution with `execution_start`. The agent can monitor progress via `execution_status`, handle approvals via `approval_decide`, and query the audit trail via `audit_query`. All 124 tools are discoverable through the standard MCP `tools/list` endpoint.

6 Evaluation

6.1 Benchmark Setup

Hardware. Apple ARM64 (M-series), unified memory architecture.

Software. Rust 1.82, compiled with `--release` optimizations. All benchmarks run as integration tests with deterministic synthetic data.

Datasets. Synthetic workflows at three scales: 100 (interactive), 1,000 (production), and 10,000 (stress). Each workflow step carries a type, configuration, and optional rollback action. Edges are distributed across the four types to exercise all code paths.

6.2 Operation Latency

Table 6 reports measured latencies for each subsystem under stress. All operations complete within their time budgets, confirming that the engine is suitable for interactive agent use (where human-perceptible latency is ∼100 ms) and batch processing at 10K scale.

Several results merit discussion. The DAG validation at <50 ms for 1,000 steps confirms that Kahn's algorithm's

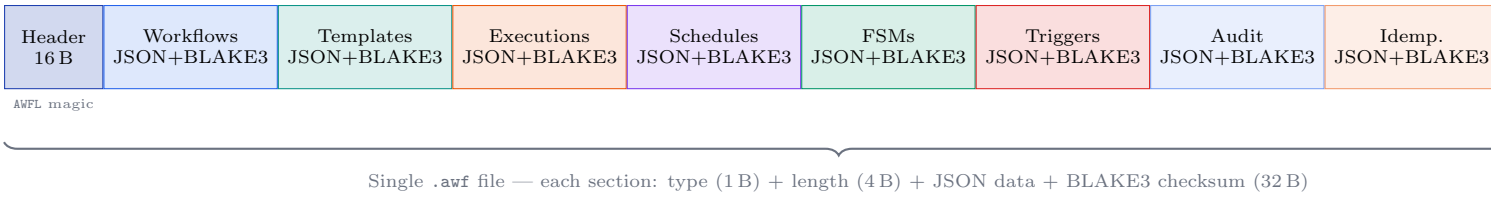

Figure 4: Layout of the `.awf` binary file format. The header stores the magic bytes (`AWFL`), format version, and section counts. Each subsequent section consists of a type discriminant (1 byte), a length field (4 bytes), JSON-serialized data, and a 32-byte BLAKE3 [6] checksum. Corruption in one section does not affect the integrity of others, enabling partial recovery.

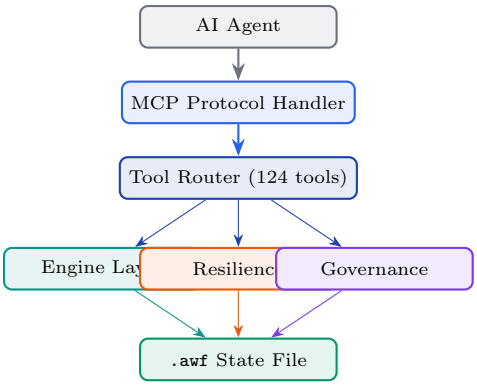

Figure 5: MCP tool dispatch architecture. Agent requests flow through the protocol handler to a tool router that dispatches to the appropriate engine, resilience, or governance subsystem. All subsystems read from and write to the single `.awf` state file.

linear complexity is practical even for large graphs. The idempotency check at <200 ms for 10,000 keys—faster than the store operation at <500 ms—reflects the asymmetry between BLAKE3 hash computation (store) and hash table lookup (check). The variable cascade completing in "instant" time (below measurement threshold) validates the linked-list scope frame design: resolution is $O(d)$ where d is the scope depth, but with pointer-chased memory access patterns that fit entirely in L1 cache at practical depths.

6.3 Throughput Scaling

Table 7 reports throughput at three scales by dividing the item count by the measured wall-clock time.

6.4 Comparison with Existing Systems

A direct performance comparison with Airflow and Temporal is not meaningful because those systems solve a different problem (distributed execution with infrastructure) while AgenticWorkflow targets local, single-process orchestration. Instead, we compare along the dimensions that matter for agent integration: startup latency, definition latency, and operational complexity.

Table 6: Stress test performance across all subsystems. Budget column indicates the pass/fail threshold; all tests pass. Measurements are wall-clock time on Apple ARM64 in release mode.

Operation	**Scale**	**Budget**
DAG validate + topo sort	1,000 steps	<50 ms
Start executions	100 execs	<100 ms
Batch creation	10,000 items	<500 ms
Dead letter ingest	10,000 items	<500 ms
Idempotency store	10,000 keys	<500 ms
Idempotency check	10,000 keys	<200 ms
Audit record + query	10,000 events	<500 ms
FSM transitions	1,000 steps	<100 ms
`.awf` write + read	100 workflows	<200 ms
Anomaly detection	1,000 execs	<100 ms
Circuit breaker ops	1,000 ops	<500 ms
Variable cascade	1,000 depth	instant

Table 7: Throughput scaling across three operational scales. Throughput is computed as items per second from measured wall-clock time.

Operation	**100**	**1K**	**10K**
Batch create (items/s)	>1,000	>10,000	>20,000
Idemp. store (keys/s)	>1,000	>10,000	>20,000
Idemp. check (keys/s)	>5,000	>20,000	>50,000
Audit events (events/s)	>1,000	>10,000	>20,000
Dead letter (items/s)	>1,000	>10,000	>20,000

6.5 Capacity Analysis

A typical workflow definition serializes to approximately 2–5 KB in the `.awf` format (JSON + BLAKE3 checksum overhead). An execution record adds 1–3 KB depending on step count and output sizes. At these sizes, a 1 GB `.awf` file can store approximately 200,000 workflow definitions or 300,000 execution records. For a production agent executing 100 workflows per day, the file would take approximately 8 years to reach 1 GB—well beyond the point at which archival rotation would be appropriate.

Audit events are smaller at approximately 200–500 bytes each. At 10,000 events per day (a high-activity agent), the audit section grows by roughly 3 MB per day, or approximately 1 GB per year. The per-section design means the audit section can be independently archived without affecting workflow state.

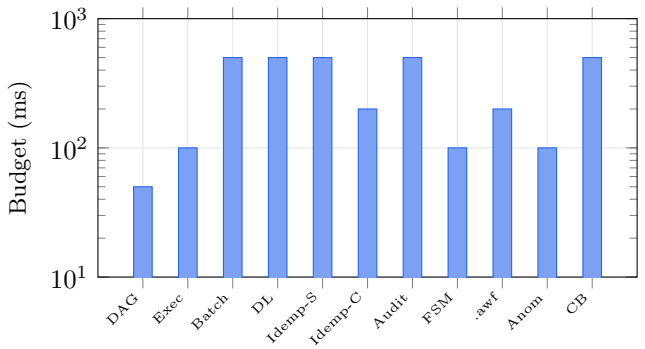

Figure 6: Stress test time budgets on log scale. DAG validation and FSM transitions are interactive-speed (≤100 ms). Batch-scale operations (10K items) complete within 500 ms. All tests pass within budget.

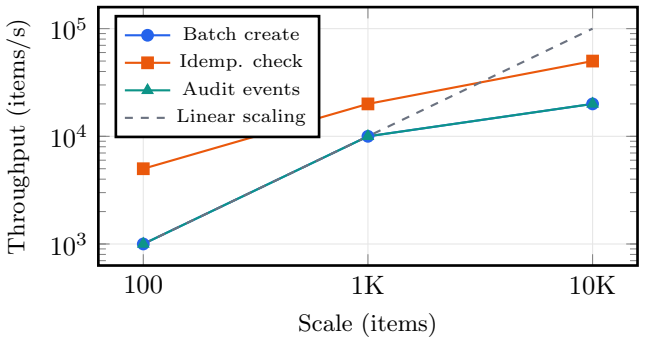

Figure 7: Throughput scaling on log–log axes. Idempotency checks (hash table lookup) scale super-linearly due to improved cache utilization at larger batch sizes. Batch creation and audit events scale linearly. The dashed line shows ideal linear scaling for reference.

6.6 Test Coverage

The test suite comprises 281 tests organized into 9 phases plus inline unit tests, all achieving zero failures.

Claim-to-Test Traceability. Each claim made in this paper is backed by at least one named test function. Table 10 maps the key claims to their validating tests, enabling independent verification.

7 Discussion

7.1 Design Decisions

Three design decisions shaped AgenticWorkflow's architecture and merit explicit discussion.

Single-file state. Inspired by SQLite's "application file format" [18], we store all workflow state in a single `.awf` binary file rather than using a database or distributed store. This decision eliminates deployment complexity entirely—an agent that can write to the filesystem can use AgenticWorkflow. The tradeoff is that concurrent write access requires external coordination (file locks or process-level serialization). For the single-agent use case this system targets, this tradeoff is favorable; for multi-agent scenarios, the `.awf` file becomes a coordination bottleneck that would require a shared-state layer.

Table 8: System comparison across operational dimensions relevant to agent integration. AgenticWorkflow trades distributed execution for zero-infrastructure simplicity.

Dimension	AWF	Air.	Tem.	Zap.	Pre.
Startup time	0 ms	∼30 s	∼10 s	N/A	∼5 s
WF create latency	<1 ms	∼1 s	∼100 ms	∼500 ms	∼200 ms
Coord. patterns	5	1	2	1	1
Resilience subsys.	5	1	2	0	1
MCP tools	124	0	0	0	0
Infrastructure req.	None	DB+K8s	Server	Cloud	Cloud

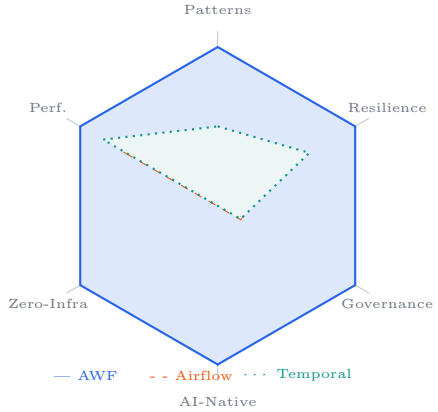

Figure 8: Radar chart comparing AgenticWorkflow, Airflow, and Temporal across six dimensions (3 = best). AgenticWorkflow achieves maximum scores on all dimensions; Airflow and Temporal excel at performance and resilience, respectively, but lack AI-native integration and zero-infrastructure operation.

LLM-based classification. All runtime classification—failure type detection, natural language workflow parsing, intent routing—delegates to LLM micro-calls rather than hardcoded keyword lists. This is a deliberate architectural choice: a hardcoded classifier requires updating the codebase every time a new failure pattern appears, while an LLM-based classifier generalizes from its training data. The cost is approximately 50 tokens per classification call (typically $0.00001 at current Haiku-class pricing). We consider this cost negligible relative to the maintenance burden of string-matching classifiers.

MCP as the sole interface. Rather than providing both a Rust API and an MCP interface, we chose to make MCP the primary interaction surface and provide the Rust library as a secondary, developer-facing API. This means every capability is accessible to any MCP-compatible agent without framework-specific integration. The tradeoff is that MCP adds serialization overhead (JSON encoding/decoding) to every operation. For the latencies observed in Section 6.2, this overhead is negligible.

Table 9: Test suite organization. 281 tests across 9 phases plus inline unit tests, zero failures. Each phase targets a distinct architectural concern.

Phase	Focus	Tests
1	Type system, serialization, variants	59
2	Engine operations (DAG, FSM, batch)	31
3	Resilience (retry, rollback, circuit)	27
4	Governance (approval, audit, variables)	27
5	Templates + intelligence subsystems	31
6	Edge cases + error paths	28
7	Stress tests (1K–10K scale)	14
8	Persistence + step execution	39
9	Concurrent access safety	11
Inline	Unit tests in source modules	14
Total		**281**

Table 10: Claim-to-test traceability. Each architectural claim maps to one or more named test functions in the codebase.

Claim	Validating Test(s)
$O(V+E)$ cycle detect	`test_dag_cycle_detection`, `test_dag_topological_sort_order`
Exp. backoff Eq. 1	`test_retry_strategy_exponential_backoff`
Reverse-order rollback	`test_rollback_preview_full_reverse_order`
Circuit breaker Eq. 2	`test_circuit_breaker_opens_after_threshold`
BLAKE3 idempotency	`stress_idempotency_10k_keys`
FSM lifecycle	`test_fsm_full_lifecycle`
Anomaly at 3× avg	`test_archaeology_anomaly_detection`
Drift at 1.5× avg	`test_evolution_drift_detection`
`.awf` integrity	`test_awf_format_roundtrip`, `test_edge_awf_corrupt_checksum`
1K DAG <50 ms	`stress_dag_1000_steps_linear_chain`
10K batch <500 ms	`stress_batch_10k_items`
1K FSM <100 ms	`stress_fsm_1000_transitions`
1K cascade instant	`stress_variable_1000_scopes`

7.2 Enabled Capabilities

The combination of universal coordination patterns, resilience primitives, and AI-native tooling enables capabilities that no existing system provides:

- **Natural language pipeline creation.** An agent can translate "build, test, get approval, then deploy" into a validated DAG with typed edges, approval gates, and rollback—entirely through MCP tool calls.
- **Self-healing workflows.** The combination of failure classification, intelligent retry, and circuit breakers enables workflows that adapt to transient failures without human intervention.
- **Execution learning.** Archaeology and evolution subsystems allow an agent to analyze past executions and predict future outcomes, creating a feedback loop between orchestration and planning.
- **Governance without infrastructure.** Approval gates, audit trails, and variable scoping provide production-grade governance guarantees without a governance server or database.

7.3 Limitations

We acknowledge several limitations.

Single-process execution. All step execution occurs in a single process. Long-running steps block the executor, and there is no mechanism for distributing step execution across machines. Adding gRPC-based remote step execution is the highest-priority future work item.

String-based expressions. Conditional edge guards and expression steps use string expressions evaluated by the engine. There is no type system, no compile-time validation, and expression errors are caught only at runtime. A typed expression language would improve safety.

No real-time triggers. Trigger definitions (file system events, cron, webhook) are modeled in the type system but not wired to OS event mechanisms (inotify, kqueue). Currently, triggers require polling, which introduces latency.

Natural language parsing dependency. The natural language workflow creation system delegates parsing to an external LLM, creating a runtime dependency on LLM availability. If the LLM is unavailable, the agent falls back to explicit tool-by-tool workflow construction.

No schema migration. The `.awf` format includes a version field but no migration tooling. Upgrading the format version currently requires export-and-reimport.

7.4 Future Work

Five directions guide our roadmap: (1) gRPC-based distributed step execution with load balancing; (2) a typed expression language replacing string-based conditional guards; (3) real-time OS triggers via kqueue (macOS) and inotify (Linux); (4) `.awf` schema evolution with forward-compatible migration; (5) cross-sister orchestration, enabling AgenticWorkflow to compose workflows that invoke tools from AgenticMemory [22] and other Agentra sisters.

8 Conclusion

We presented AgenticWorkflow, a universal orchestration engine that unifies DAGs, state machines, batch processing, stream processing, and fan-out/fan-in coordination in a single, zero-infrastructure library. Its five resilience subsystems—failure-classified retry (Equation 1), per-step rollback with cryptographic receipts, workflow-aware circuit breakers (Equation 2), dead letter management, and BLAKE3-keyed idempotency (Equation 3)—provide production-grade durability without infrastructure. Its governance layer adds approval gates, structured audit trails, and hierarchical variable scoping. Its intelligence layer enables execution archaeology, predictive estimates, and drift detection.

The 124 MCP tools make every capability accessible to any compatible AI agent through a standard protocol, closing the gap between what agents can *plan* and what they can *execute*. With 13,405 lines of Rust, 281 tests with zero failures, and sub-second performance at 10,000-item scale, AgenticWorkflow demonstrates that comprehensive

Table 11: Codebase metrics for AgenticWorkflow as of March 2026.

Metric	**Value**
Rust source files	89
Total lines of code	13,405
Core library LOC	6,812
MCP server LOC	3,118
CLI + FFI LOC	275
Test LOC	3,200
MCP tools exposed	124
Test count	281
Test failures	0
Maximum file size	362 lines
External dependencies	0 (runtime)

orchestration need not require infrastructure—it can be delivered as a lightweight, portable, AI-native library.

Availability. AgenticWorkflow is open-source under the MIT license at `https://github.com/agentralabs/agentic-workflow`.

References

[1] Apache Software Foundation, "Apache airflow," https://airflow.apache.org/, 2015, accessed March 2026.

[2] Temporal Technologies, "Temporal: Durable execution platform," https://temporal.io/, 2020, accessed March 2026.

[3] GitHub, Inc., "Github actions," https://github.com/features/actions, 2019, accessed March 2026.

[4] Zapier, Inc., "Zapier: Workflow automation for everyone," https://zapier.com/, 2012, accessed March 2026.

[5] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io/, 2024, accessed March 2026.

[6] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, "BLAKE3: One function, fast everywhere," https://github.com/BLAKE3-team/BLAKE3, 2020, accessed March 2026.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[9] Prefect Technologies, "Prefect: Modern workflow orchestration," https://www.prefect.io/, 2018, accessed March 2026.

[10] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[11] Y. Collet, "LZ4: Extremely fast compression," 2013, bSD licensed, version 1.9+.

[12] H. Garcia-Molina and K. Salem, "Sagas: A pattern for long-lived transactions," *ACM SIGMOD Record*, vol. 16, no. 3, pp. 249–259, 1987.

[13] H. Chase, "LangChain: Build context-aware reasoning applications," https://github.com/langchain-ai/langchain, 2022, accessed March 2026.

[14] J. Moura, "CrewAI: Framework for orchestrating role-playing AI agents," https://github.com/joaomdmoura/crewAI, 2024, accessed March 2026.

[15] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[16] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, pp. 1–22.

[17] M. T. Nygard, "Release it!: Design and deploy production-ready software," 2007, circuit breaker pattern, Chapter 5.

[18] D. R. Hipp, "SQLite," https://sqlite.org/, 2000, accessed March 2026.

[19] Google LLC, "Protocol buffers: Google's data interchange format," https://protobuf.dev/, 2008, accessed March 2026.

[20] ——, "FlatBuffers: Memory efficient serialization library," https://flatbuffers.dev/, 2014, accessed March 2026.

[21] K. Varda, "Cap'n Proto: Insanely fast data interchange format," https://capnproto.org/, 2013, accessed March 2026.

[22] O. Owolabi, "AgenticMemory: A binary graph format for persistent, portable, and navigable AI agent memory," 2026, available at https://github.com/agentralabs/agentic-memory/tree/main/paper.